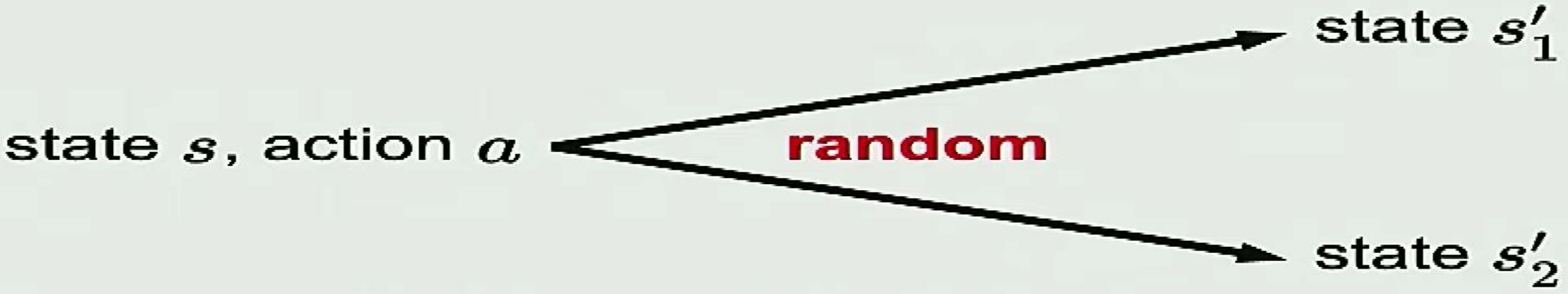
Reinforcement Learning with Agents

Dr. Kaneeka Vidanage (PhD.)

Uncertainty in the real world





Applications



Robotics: decide where to move, but actuators can fail, hit unseen obstacles, etc.



Resource allocation: decide what to produce, don't know the customer demand for various products



Agriculture: decide what to plant, but don't know weather and thus crop yield

Terminology

First things first, before even starting with MDPs, we'll quickly glance through the terminology that will be used throughout this article:

1. **Agent:** An RL agent is the entity which we are training to make correct decisions (for eg: a Robot that is being trained to move around a house without crashing).
2. **Environment:** The environment is the surrounding with which the agent interacts (for eg: the house where the Robot moves). The agent cannot manipulate the environment; it can only control its own actions (for eg: the Robot cannot control where a table is kept in the house, but it can walk around it to avoid crashing).
3. **State:** The state defines the current situation of the agent (for eg: it can be the exact position of the Robot in the house, or the alignment of its two legs, or its current posture; it depends on how you address the problem).
4. **Action:** The choice that the agent makes at the current time step (for eg: it can move its right or left leg, or raise its arm, or lift an object, turn right or left, etc.). We know the set of actions (decisions) that the agent can perform in advance.
5. **Policy:** A policy is the thought process behind picking an action. In practice, it is a probability distribution assigned to the set of actions. Highly rewarding actions will have a high probability and vice versa.

Note that if an action has a low probability, it doesn't mean it won't be picked at all. Just that it is less likely to be picked.

Markov Property

A state S_t is *Markov* if and only if

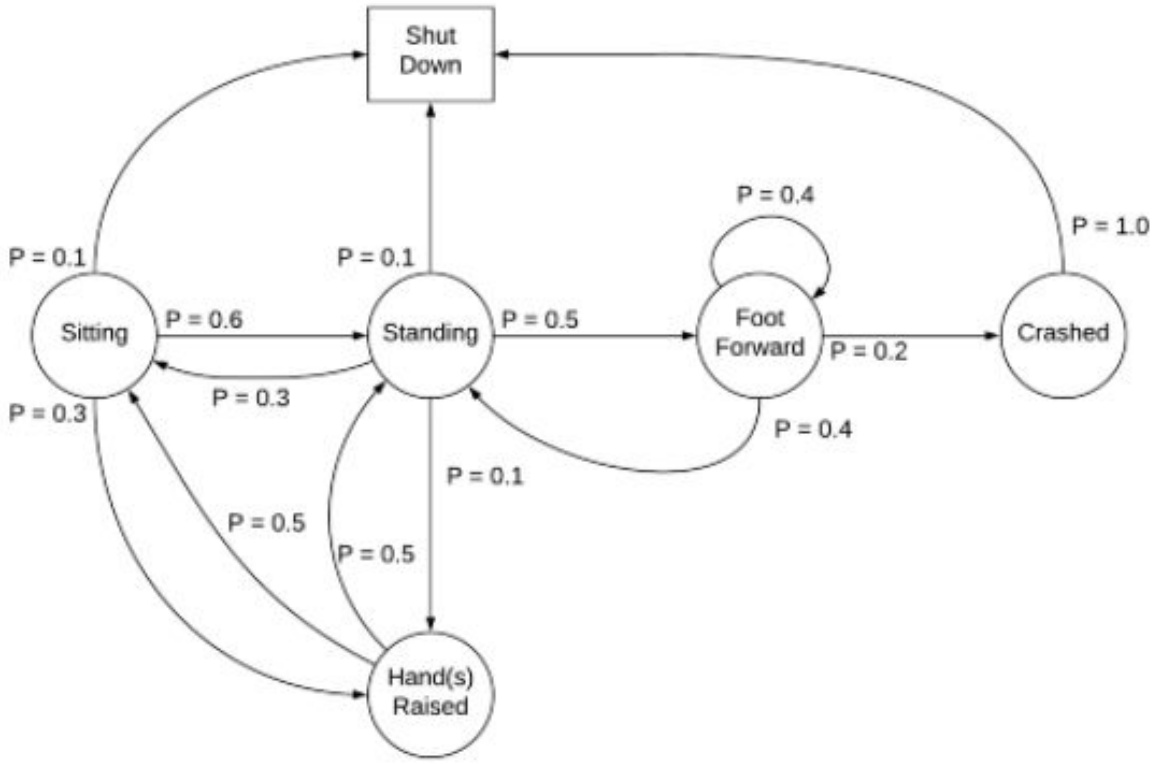
$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \ldots, S_t]$$

Markov Property

Assume that a Robot was seated on a chair, it stood up and put its right foot forward. So currently, it is standing with its right foot forward (this is its current state).

Now, according to the Markov Property, the current state of the Robot depends only on its immediate previous state (or the previous timestep) i.e., the state it was in when it stood up. And evidently, it doesn't depend on the state where it was sitting on the chair. Similarly, its next state depends only on its current state.

All historical states does not matter, only the immediate previous



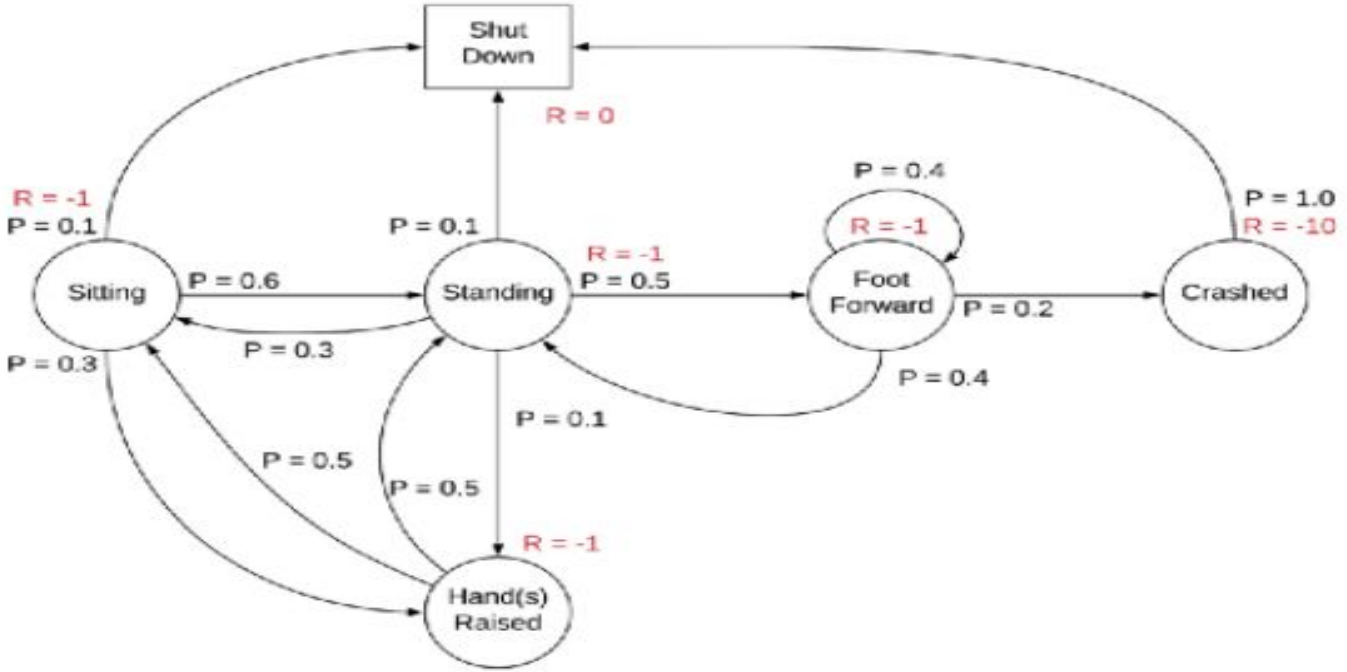
A Sample Markov Chain for the Robot Example

To get an intuition of the concept, consider the figure above. Sitting, Standing, Crashed, etc. are the states, and their respective state transition probabilities are given.

An MRP is defined by ***(S, P, R, γ)***, where ***S*** are the states, ***P*** is the state-transition probability, ***R_s*** is the reward, and ***γ*** is the discount factor (will be covered in the coming sections).

The state reward ***R_s*** is the **expected reward** over all the possible states that one can transition to from state ***s***. This reward is received for being at the state ***S_t***. **By convention**, it is said to be received after the agent leaves the state and hence, regarded as ***R_(t+1)***.

For example:



A Simple MRP Example

An MDP is defined by (S, A, P, R, γ), where A is the set of actions. It is essentially **MRP with actions.** Introduction to actions elicits a notion of control over the Markov Process, i.e., previously, the state transition probability and the state rewards were more or less stochastic (random). However, now the rewards and the next state also depend on what action the agent picks. Basically, the agent can now control its own fate (to some extent).

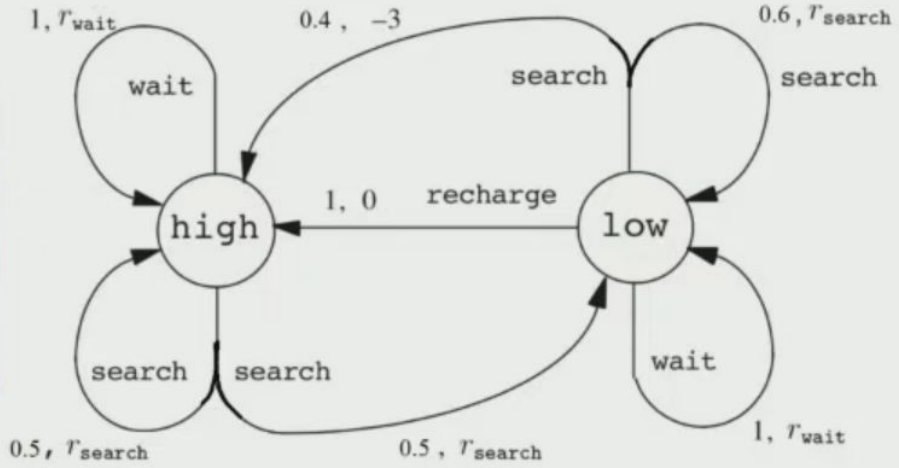
Markov Decision Process

Goal. Develop a unified language to specify models, policies and value functions.

Example. A robot that searches for items.

- The robot is battery-powered and can be in two states: high, low.
- The robot either waits or searches. A third alternative is to recharge if low power.
- If the robot continues to search, with certain probability it will power-off and need costly manual restart.





Definition

A Markov decision process (MDP) is a 4-tuple $M = (S, A, P, R, \gamma)$ where

- the set S is the states space,
- the set A is the action space,
- $P\colon S \times A \times S \to [0,1]$ is a transition probability function,

  $P(s,a,s')$ is the probability of a transition from state s to s' with action a, and is denoted by $P^a_{s \to s'}$.
- $R\colon S \times A \times S \to \mathbb{R}$ is a (state-based) reward function.

  $R(s,a,s')$ is the (immediate) reward the agent receives from a transition from s to s' with action a, and is denoted by $R^a_{s \to s'}$.
- $\gamma \in (0,1)$ is a discount factor.

Definition

A play (or run) of an MDP is a (finite or infinite) sequence $s_0, a_1, r_1, s_1, a_2, r_2, s_2, a_3, r_3, s_3, \ldots$ such that every $s_i \in S$, every $a_i \in A$, and $r_{i+1} = R^{a_{i+1}}_{s_i \to s_{i+1}}$.

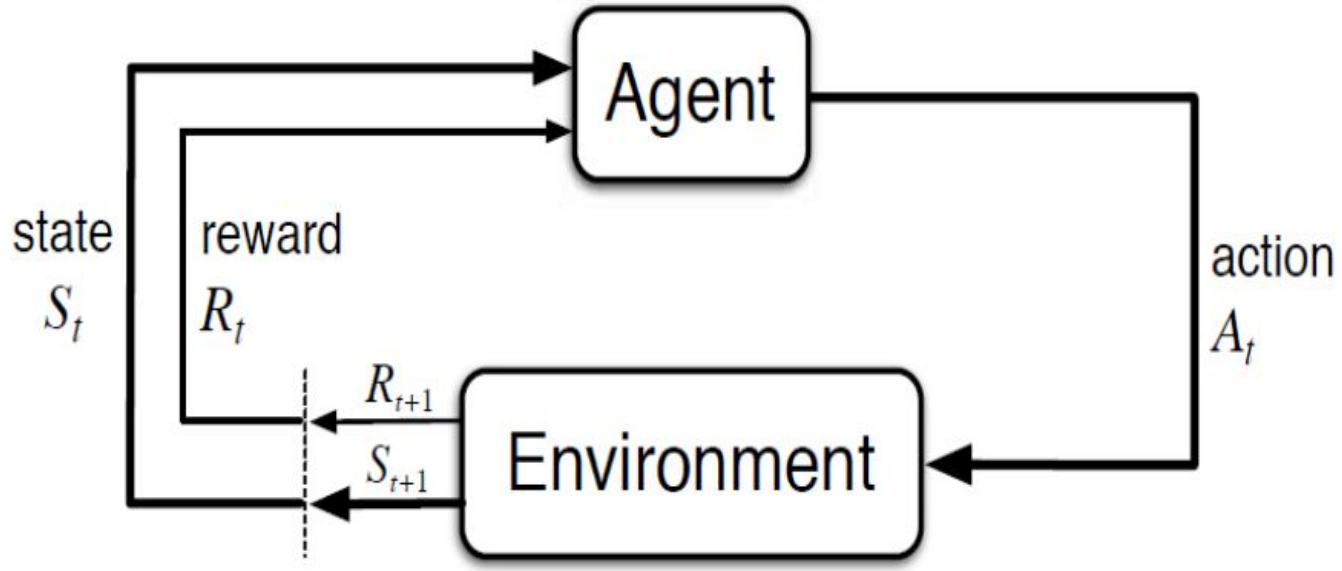
Example.

- $S = \{H, L\}$ and $A = \{\texttt{search}, \texttt{wait}, \texttt{recharge}\}$
- $P^a_{s \to s'}$ and $R^a_{s \to s'}$ are given by the table below:

s	a	s'	$p(s,a,s')$	$r(s,a,s')$
high	search	high	0.5	$r_{\texttt{search}}$
high	search	low	0.5	$r_{\texttt{search}}$
low	search	high	0.4	-3
low	search	low	0.6	$r_{\texttt{search}}$
high	wait	high	1	$r_{\texttt{wait}}$
high	wait	low	0	$r_{\texttt{wait}}$
low	wait	high	0	$r_{\texttt{wait}}$
low	wait	low	1	$r_{\texttt{wait}}$
low	recharge	high	1	0
low	recharge	low	0	0.

A play: high, wait, $r_{\texttt{wait}}$, high, search, $r_{\texttt{search}}$, low, recharge, 0, high

Markov Decision Process (MDP) is a mathematical framework to describe an environment in reinforcement learning. The following figure shows agent-environment interaction in MDP:



More specifically, the agent and the environment interact at each discrete time step, t = 0, 1, 2, 3...At each time step, the agent gets information about the environment state S_t. Based on the environment state at instant t, the agent chooses an action A_t. In the following instant, the agent also receives a numerical reward signal R_{t+1}. This thus gives rise to a sequence like S_0, A_0, R_1, S_1, A_1, R_2...

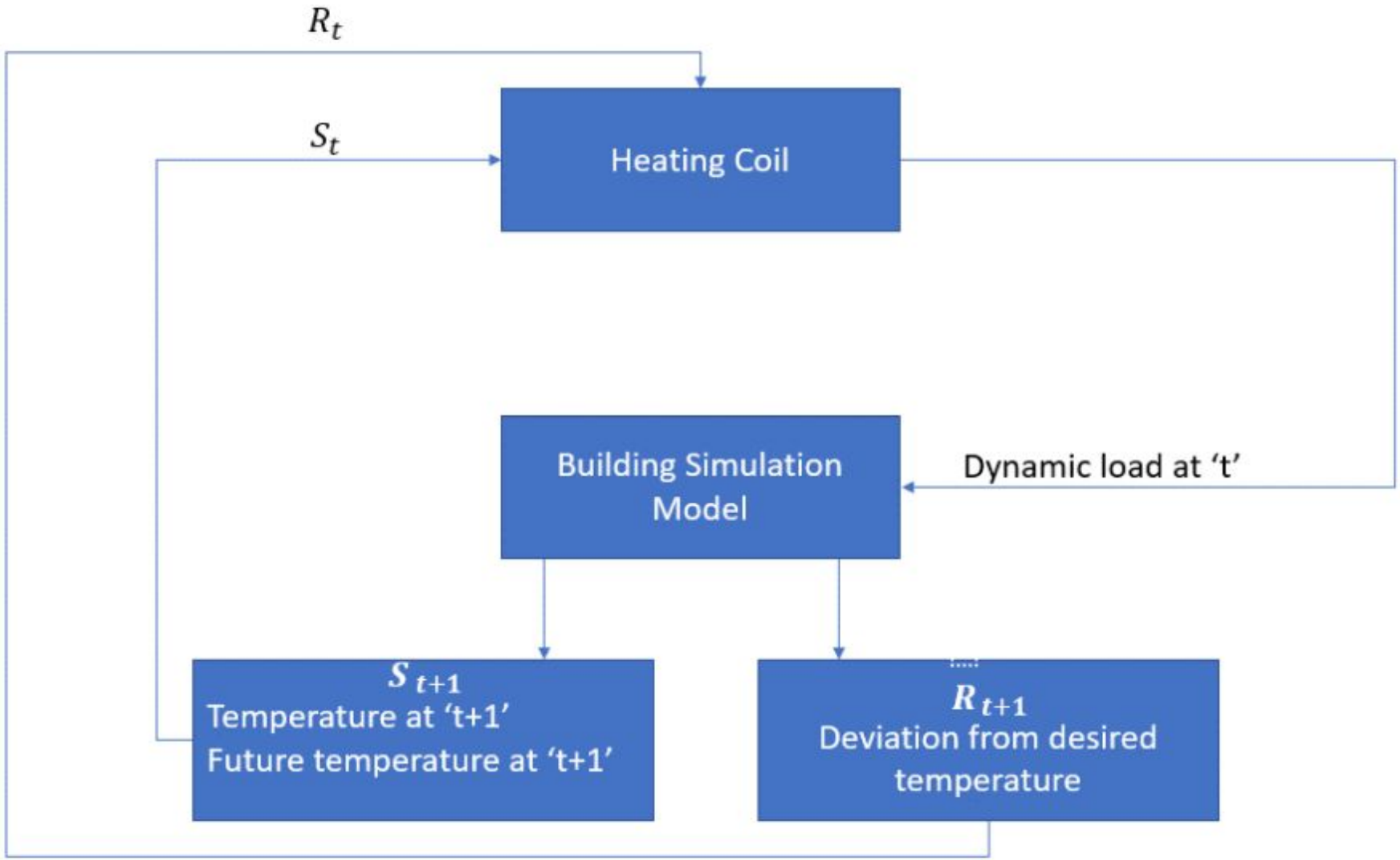
Let us now discuss a simple example where RL can be used to implement a control strategy for a heating process.

The idea is to control the temperature of a room within the specified temperature limits. The temperature inside the room is influenced by external factors such as outside temperature, the internal heat generated, etc.

The agent, in this case, is the heating coil which has to decide the amount of heat required to control the temperature inside the room by interacting with the environment and ensure that the temperature inside the room is within the specified range. The reward, in this case, is basically the cost paid for deviating from the optimal temperature limits.

The action for the agent is the dynamic load. This dynamic load is then fed to the room simulator which is basically a heat transfer model that calculates the temperature based on the dynamic load. So, in this case, the environment is the simulation model. The state variable S_t contains the present as well as future rewards.

The following block diagram explains how MDP can be used for controlling the temperature inside a room:


R_t
S_t
Heating Coil
Building Simulation Model
Dynamic load at 't'
S_{t+1}
Temperature at 't+1'
Future temperature at 't+1'
R_{t+1}
Deviation from desired temperature

The Bellman Equation

Concepts:

- s – State
- a – Action
- R – Reward
- γ – Discount

The Bellman Equation

Concepts:

- s – State
- a – Action
- R – Reward
- γ – Discount

Discount Factor [DF] reduces the value of state, when you are away from the targeted goal. This rationale of discount factor provides lots of control in by preventing the agent going across wrong paths, as when reaching towards the goal, state`s value will get increased.

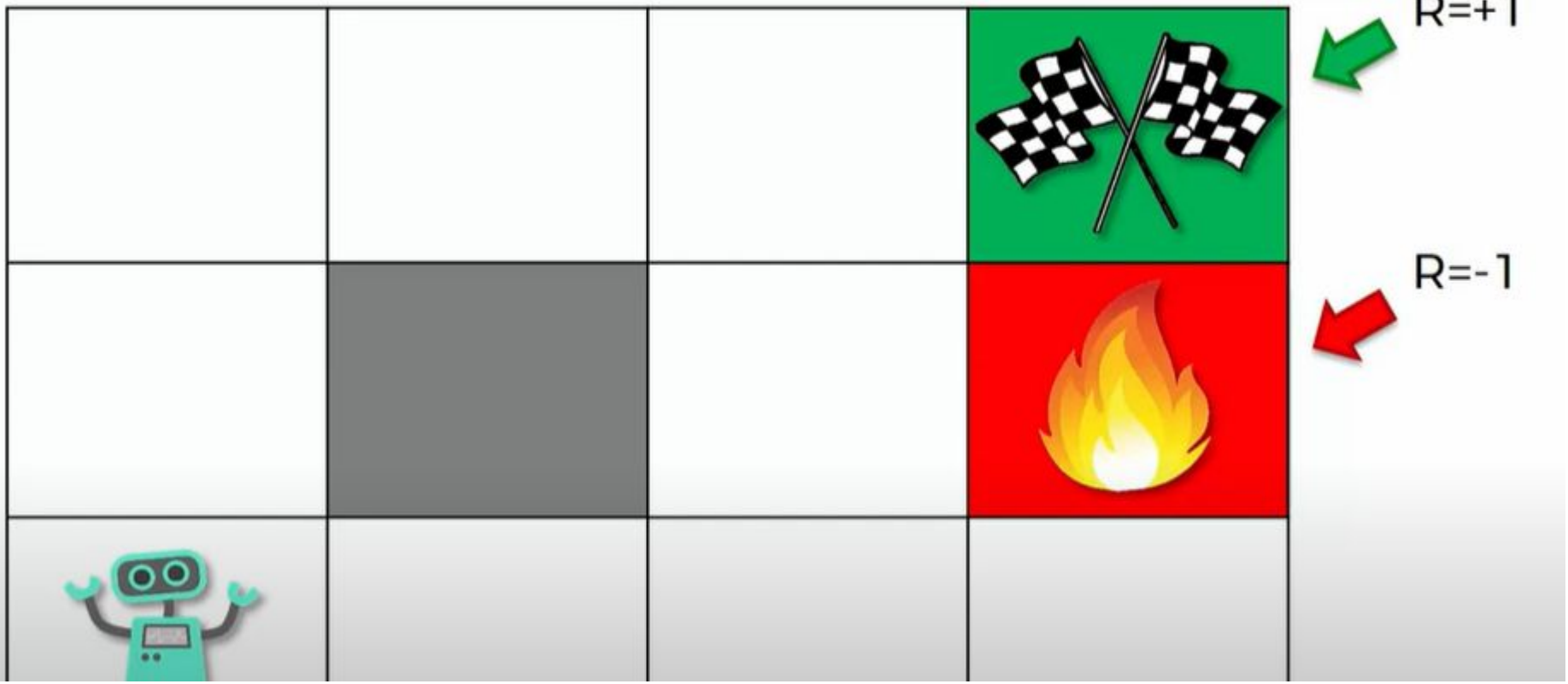
The Bellman Equation

- The Bellman equation can be written as:

$$V(s) = \max[R(s,a) + \gamma V(s')]$$

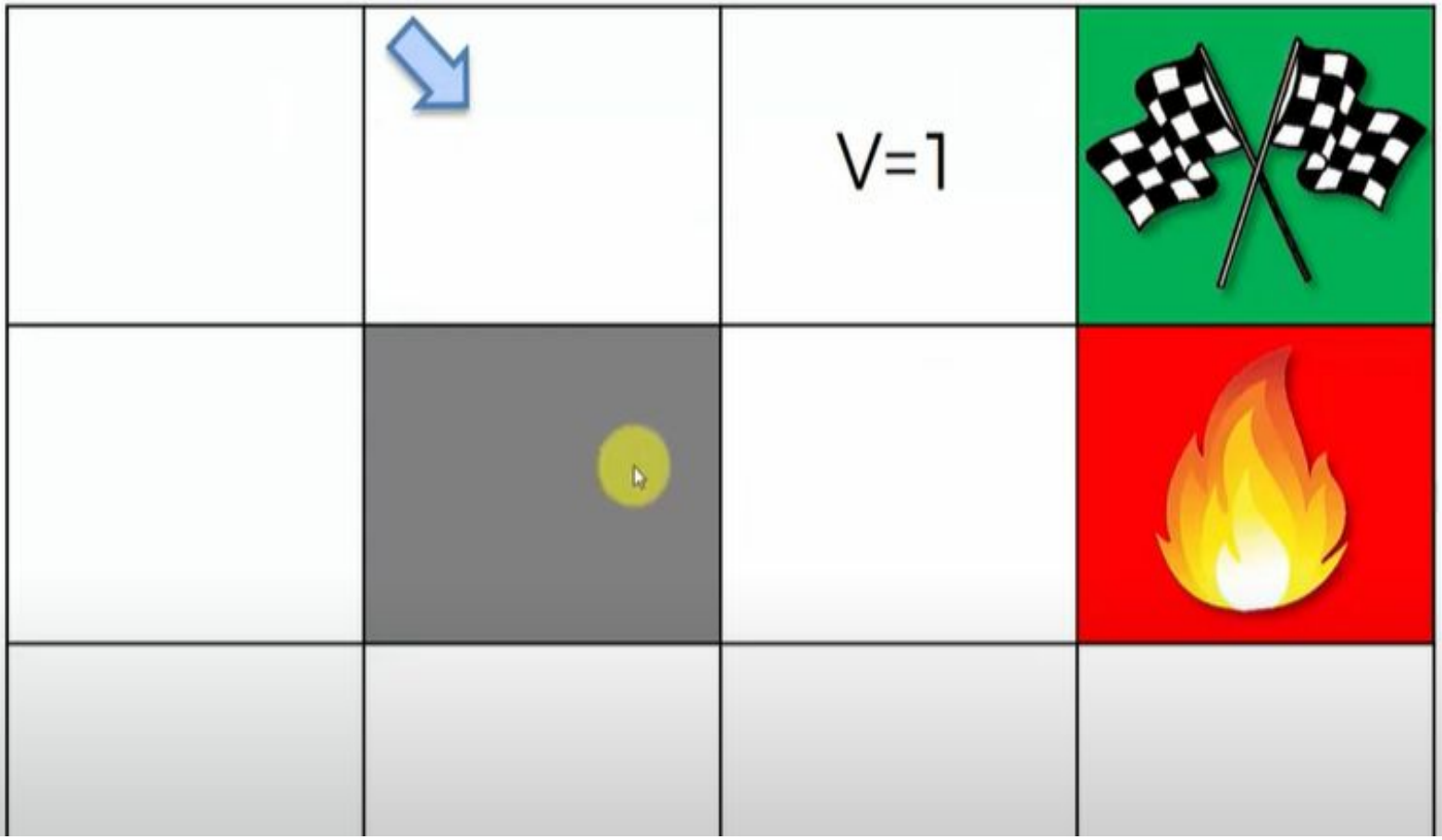
- Where,
- $V(s) = value\ calculated\ at\ a\ particular\ point.$ **New state □ S**
- $R(s,a) = Reward\ at\ a\ particular\ state\ s\ by\ performing\ an\ action\ a.$
- $\gamma = Discount\ factor$
- $V(s') = The\ value\ at\ the\ previous\ state.$ **Old state □ S^1**
- In the above equation, we are taking the max of the complete values because the agent tries to find the optimal solution always.

The Bellman Equation



The Bellman Equation

$$V(s) = \max_a(R(s,a) + \gamma V(s'))$$



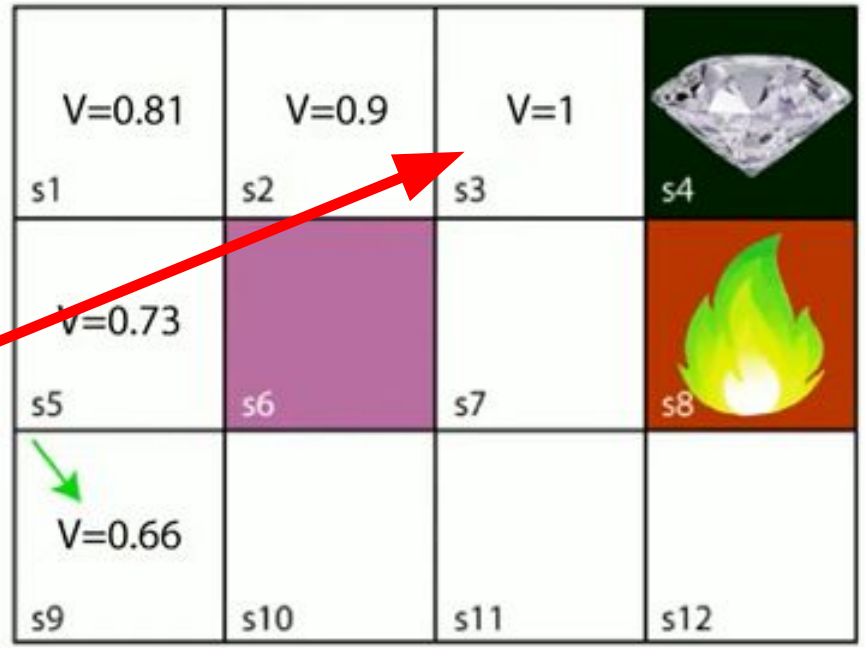
$\gamma = 0.9$

The Bellman Equation

- So now, using the Bellman equation, we will find value at each state of the given environment.

- We will start from the block, which is next to the target block.

- **For S3 block:**
- $V(s3) = \max[R(s,a) + \gamma V(s`)]$,
- here $V(s') = 0$ because there is no further state to move.
- $V(s3) = \max[R(s,a)] => V(s3) = \max[1] => V(s3) = 1.$

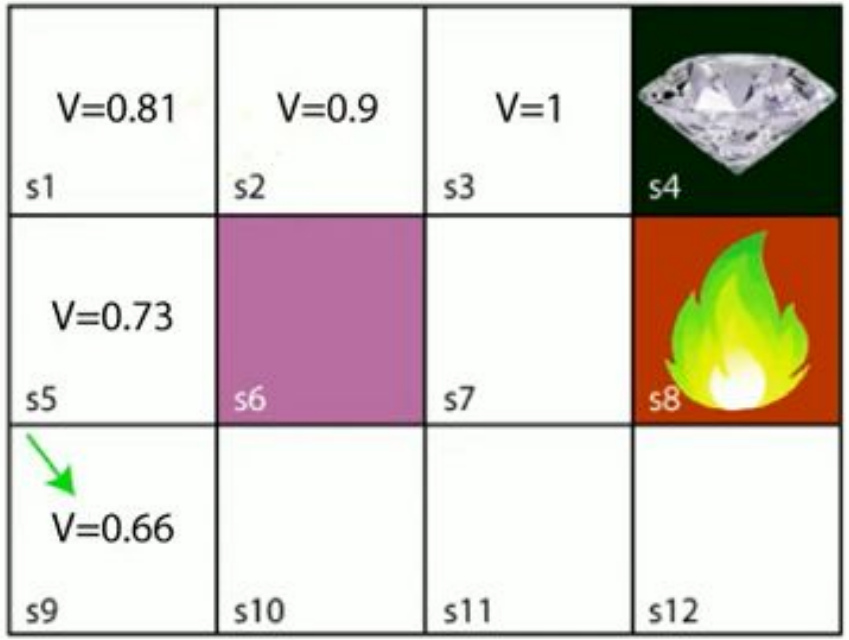


- Bellman equation performs it`s calculations from goal towards the starting point **in a backward fashion**. (i.e., s4□s3□s2□s1)
- For s3 block, prev state is s4, which is the goal state. Consider the goal state as state 0. That's why V(s^1) becomes o in here.
- Here, the R(s,a) returns 1. Becz, it`s the reward given for s3 □s4 transition, where the s4 is the goal with maximum reward (R) of 1.

The Bellman Equation

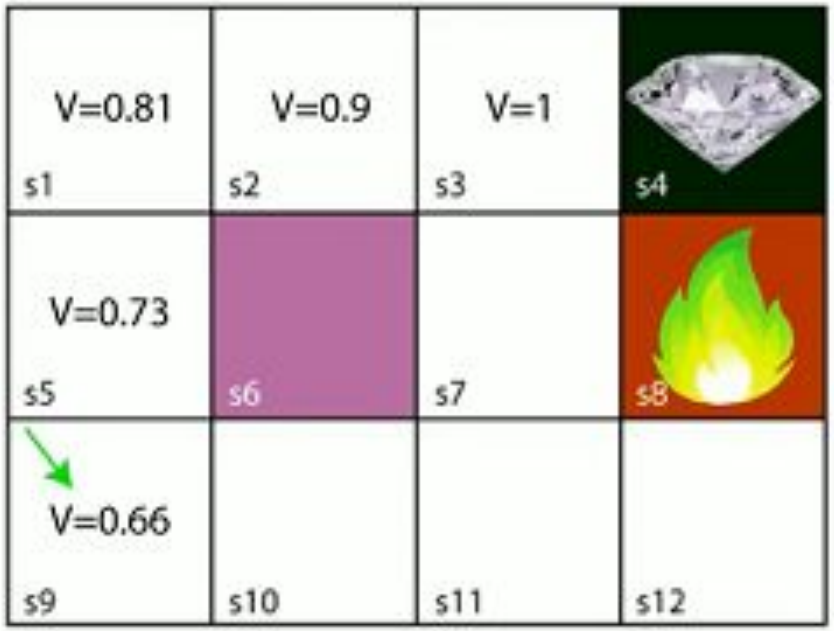
- **For S2 block:**
- $V(s2) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9, V(s') = 1, and\ R(s,a) = 0$, because there is no reward at this state.
- $V(s2) = \max[0.9(1)] => V(s2) = \max[0.9] => V(s2) = 0.9$

- Here, the prev state $V(s^1)$ =1 as we calculated it in earlier slide.
- DF is assumed as 0.9 theoretically.
- Here, R becomes 0, as there is no reward for moving from s2□s3, hence s3 is not a goal.
- Hence V(s2) □0.9



The Bellman Equation

- **For S1 block:**
- $V(s1) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9(lets), V(s') = 0.9, and\ R(s,a) = 0$, because there is no reward at this state also.
- $V(s1) = \max[0.9(0.9)] => V(s1) = \max[0.81] =>$ $V(s1) = 0.81$

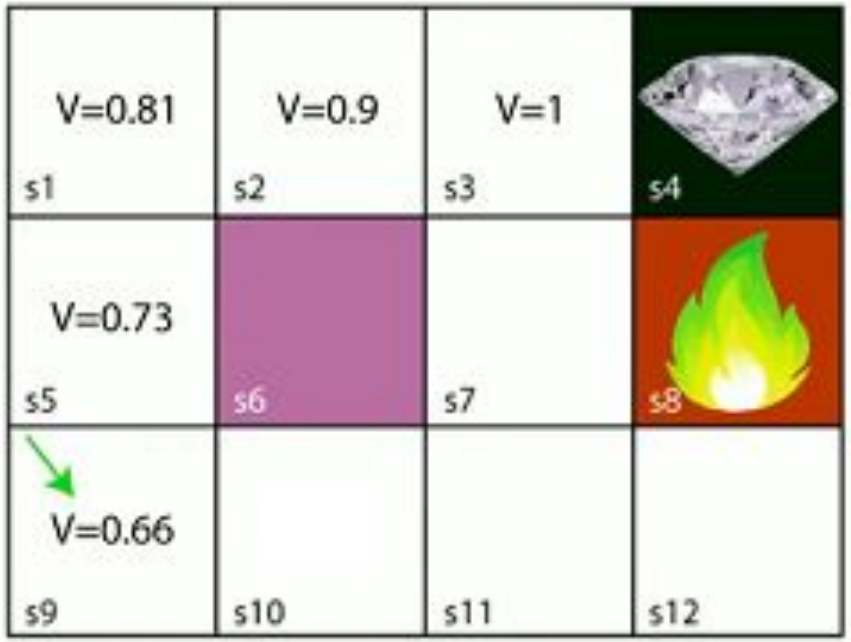


- **For S5 block:**
- $V(s5) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9(lets), V(s') = 0.81, and\ R(s,a) = 0$, because there is no reward at this state also.
- $V(s5) = \max[0.9(0.81)] => V(s5) = \max[0.73] =>$ $V(s5) = 0.73$

The Bellman Equation

- **For S9 block:**

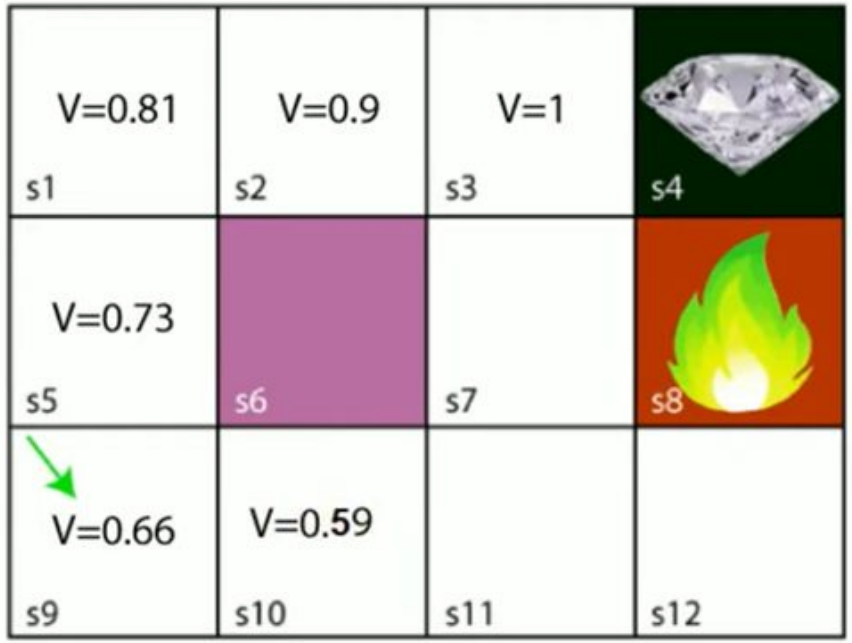
- $V(s9) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9(lets), V(s') = 0.73, and\ R(s,a) = 0$, because there is no reward at this state also.
- $V(s9) = \max[0.9(0.73)] => V(s9) = \max[0.66] => V(s4) = 0.66$



The Bellman Equation

- **For S10 block:**

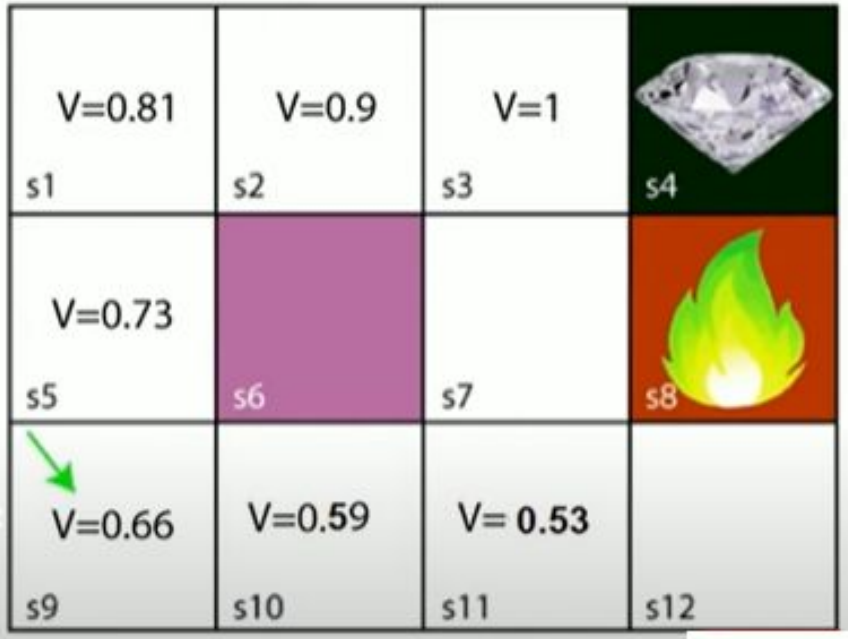
- $V(s10) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9, V(s') = 0.66$ *and* $R(s,a) = 0$, because there is no reward at this state also.
- $V(s10) = \max[0.9(0.66)] => V(s10) = \max[0.59] => V(s10) = 0.59$



The Bellman Equation

- **For S11 block:**

- $V(s11) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9, V(s') = 0.59$ *and* $R(s,a) = 0$, because there is no reward at this state also.
- $V(s11) = \max[0.9(0.59)] => V(s11) = \max[0.53] => V(s11) = 0.53$

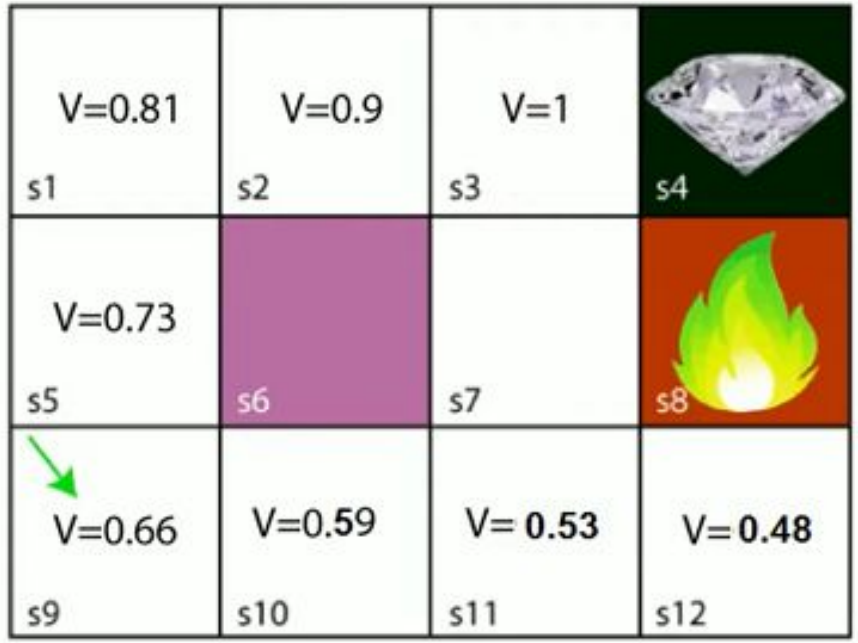


The Bellman Equation

- **For S12 block:**

- $V(s12) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9, V(s') = 0.59$ *and* $R(s,a) = 0$, because there is no reward at this state also.
- $V(s12) = \max[0.9(0.53)] => V(s12) = \max[0.48]$

  $V(s12) = 0.48$



The Bellman Equation

- **For S7 block:**

- $V(s7) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9, V(s') = 1$ *and* $R(s,a) = 0$, because there is no reward at this state also.
- $V(s7) = \max[0.9(1)] => V(s7) = \max[0.9] =>$ $V(s7) = 0.9$

V=0.81 s1	V=0.9 s2	V=1 s3	 s4
V=0.73 s5	s6	V=0.9 s7	 s8
V=0.66 s9	V=0.59 s10	V= **0.53** s11	V= **0.48** s12

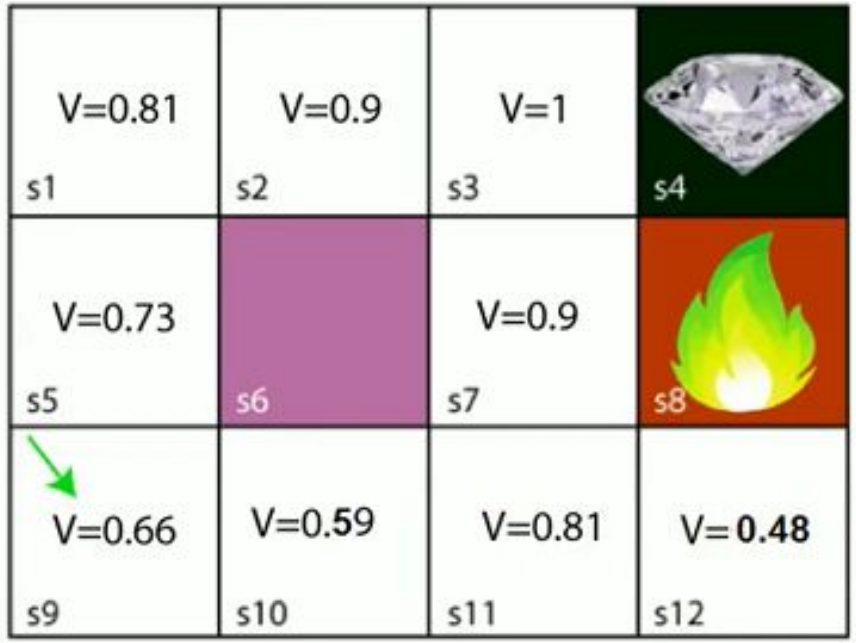
The Bellman Equation

- **For S11 block:**

s7

- $V(s11) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9, V(s') = 0.9$ *and* $R(s,a) = 0$, because there is no reward at this state also.
- $V(s11) = \max[0.9(0.9)] => V(s11) = \max[0.81] = > V(s11) = 0.81$

  - Perform a re-calculation, assuming the path s12□**s11□s7**□s3□s4
  - Compare with the prev slide. For this path we have got
    - s11 □0.81
    - Earlier path s11 □0.53

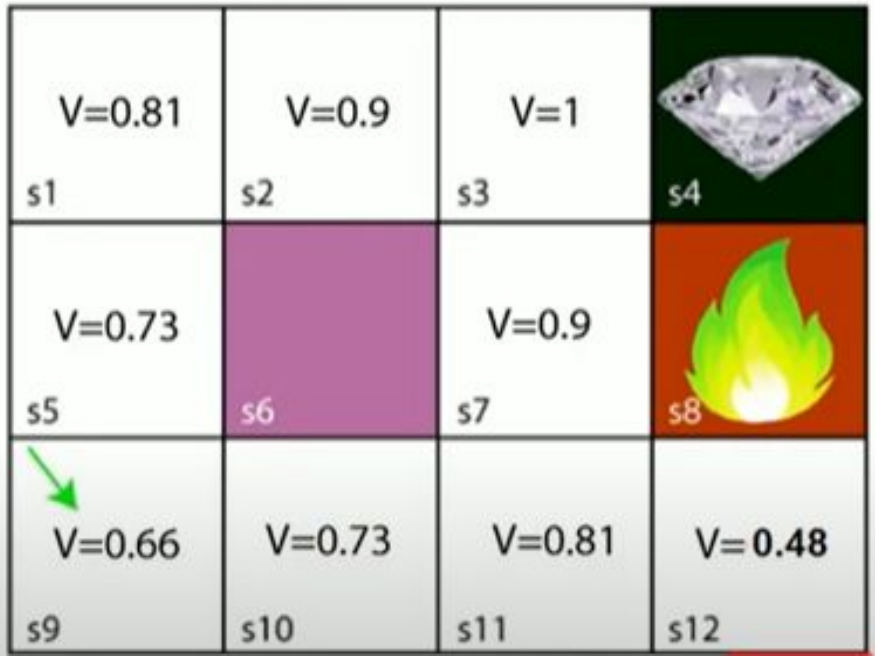


The Bellman Equation

- **For S10 block:**

Considering s11 0.81 re calculate s10
Compare with slide #26, where s10 0.59.
Hence, this path adjustment is far better

- $V(s10) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9, V(s') = 0.81$ *and* $R(s,a) = 0$, because there is no reward at this state also.
- $V(s10) = \max[0.9(0.81)] => V(s10) = \max[0.73] = > V(s10) = 0.73$

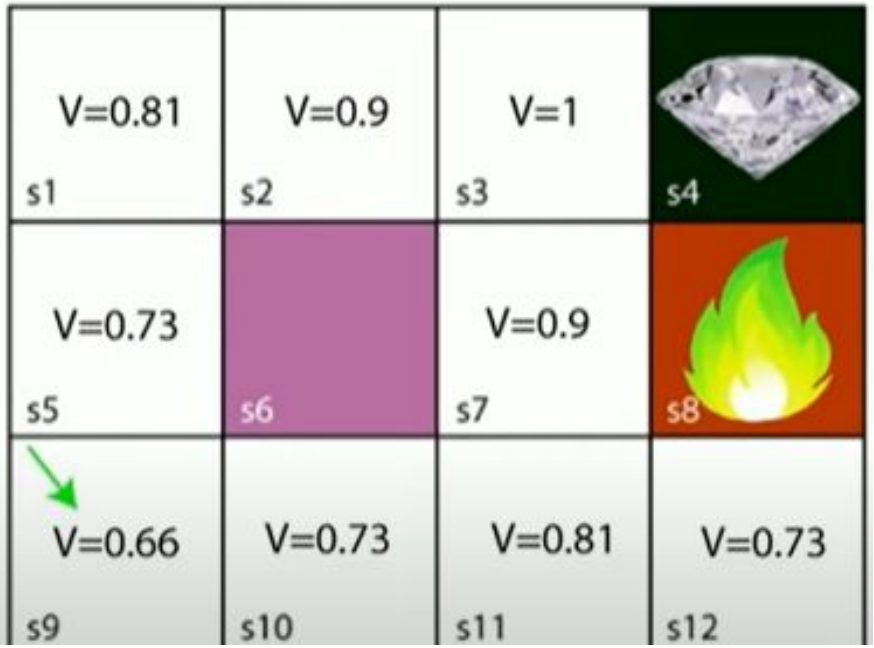


The Bellman Equation

- **For S12 block:**

Considering s11 0.81 re calculate s12 as well
Compare with slide #26, where s12 0.48.
Hence, this path adjustment is far better

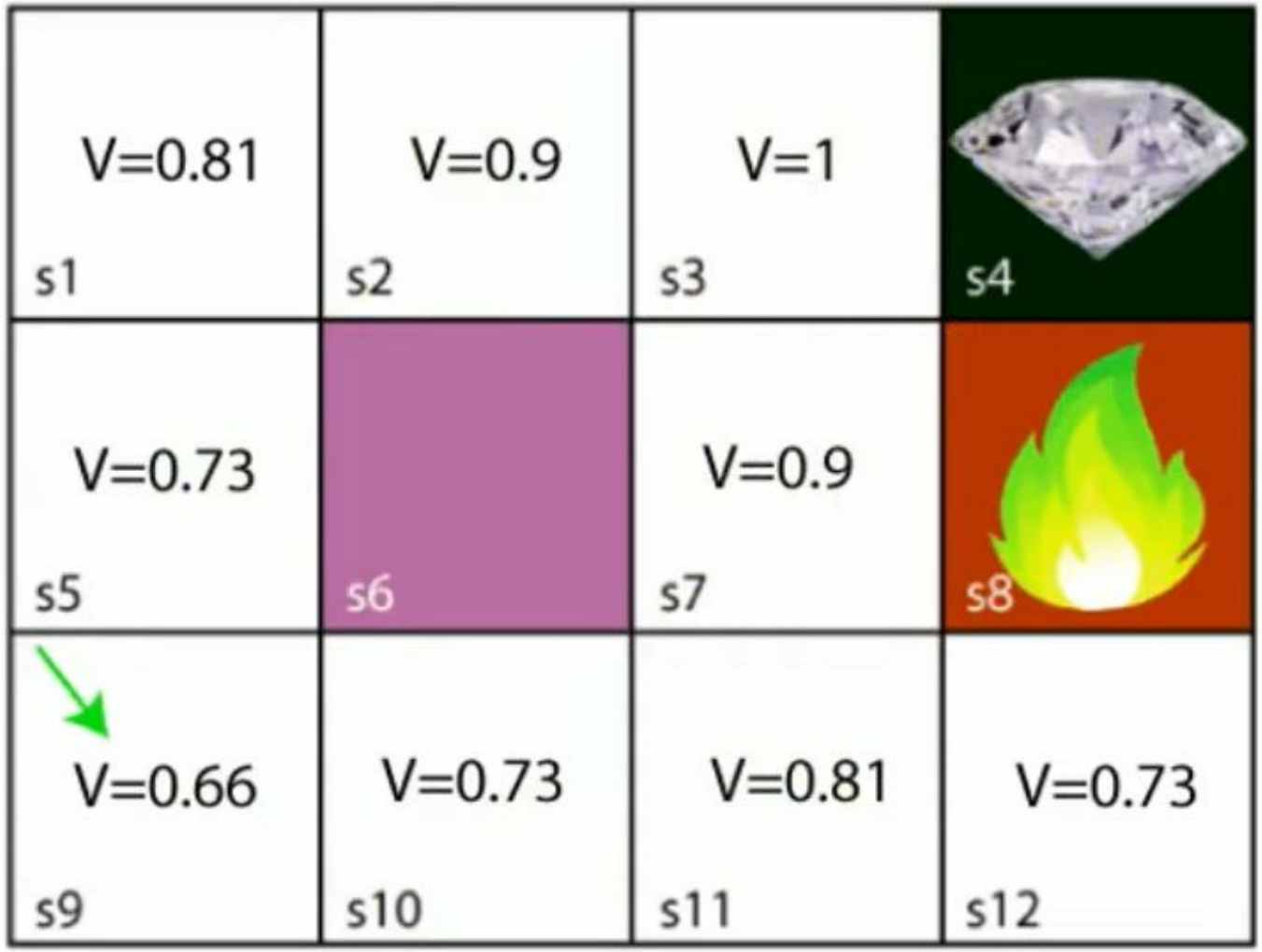
- $V(s12) = \max[R(s,a) + \gamma V(s`)]$,
- here $\gamma = 0.9, V(s') = 0.81$ *and* $R(s,a) = 0$, because there is no reward at this state also.
- $V(s12) = \max[0.9(0.81)] => V(s12) = \max[0.73] = > V(s12) = 0.73$



- **Continue this calculation for all possible paths. Finally fill the grid with max potential values. That`s why Bellman equation has max in this formulae**

Path determination with Bellman values

- Assume this is the latest grid filled with maximum Bellman values.
- Assume Agent is at s10. Via considering the Bellman values it can reach to the goal as : -
  - s10 will not go to s9 as it`s smaller
  - s10 s11 s7 s3 s3
- If Agent is at s5, then the path should be :-
  - s5 will not go to s6 as it`s smaller.
  - s5 s1 s2 s3 s4



The Bellman Equation

$$V(s) = \max_a(R(s,a) + \gamma V(s'))$$

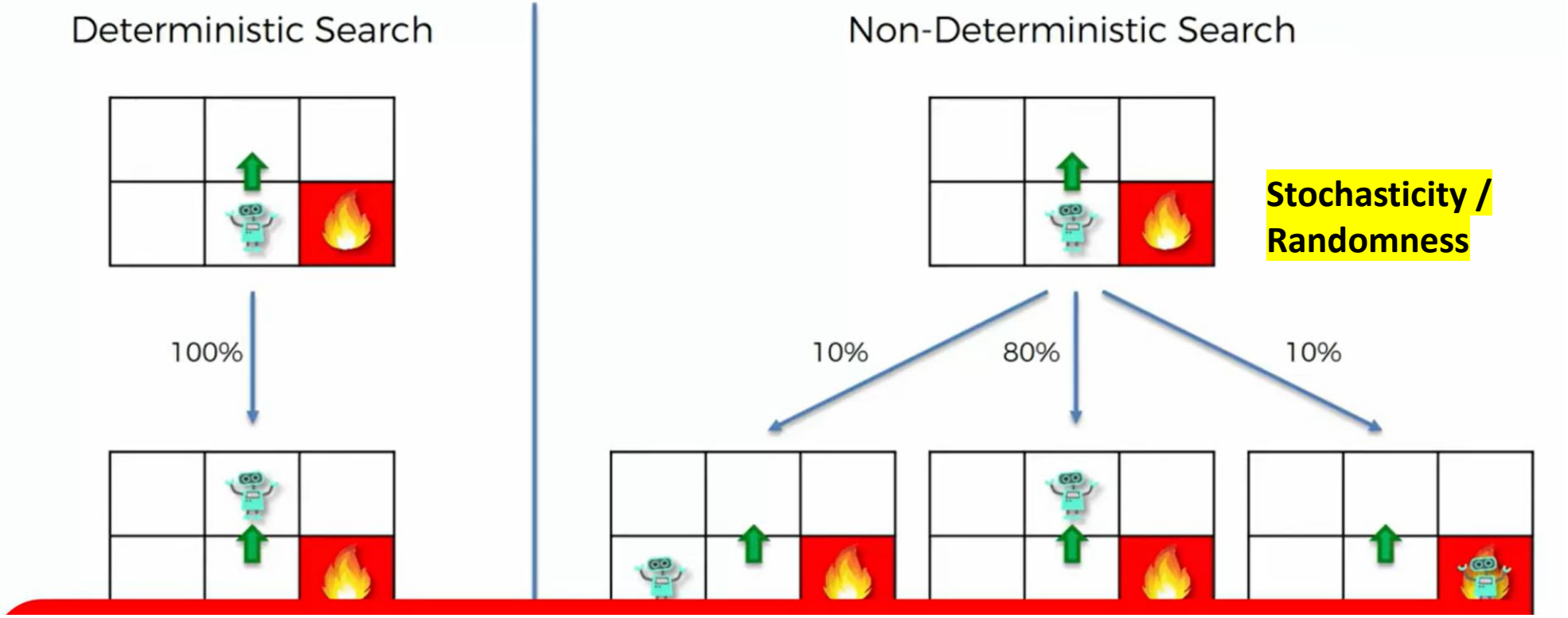
$\gamma = 0.9$

State Values



Discount factor reduces the value of state, when you are away from the targeted goal. This rationale of discount factor provides lots of control in by preventing the agent going across wrong paths, as when reaching towards the goal, state`s value will get increased.

Markov Decision Process (MDP)



Markov Decision Process (MDP)

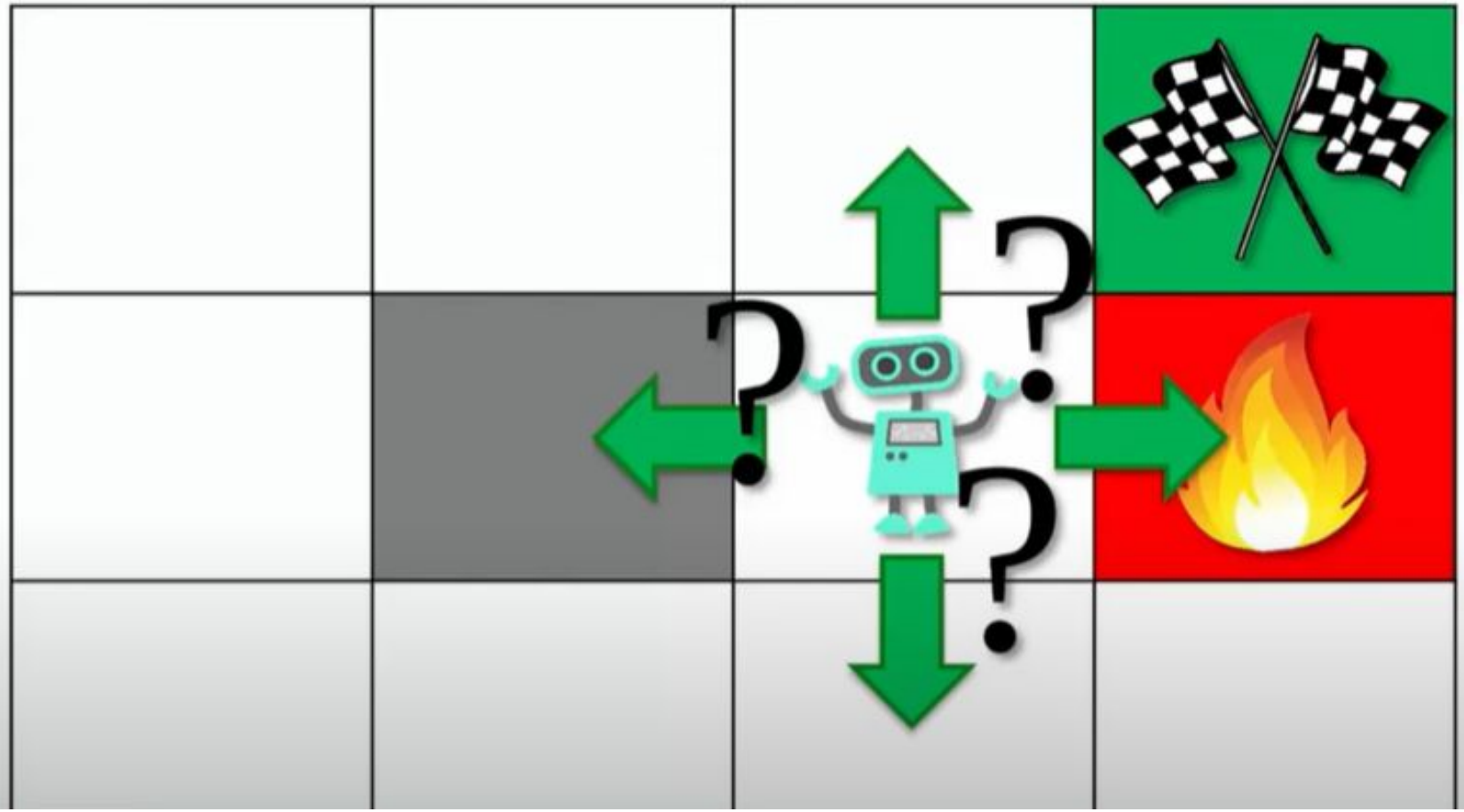
Markov Process

Markov Decision Process (MDP)

Markov Decision Process (MDP)

Markov Decision Processes (MDPs) provide a mathematical framework for modeling decision making in situations where outcomes are partly random and partly under the control of a decision maker.

Markov Decision Process (MDP)



Markov Decision Process (MDP)

$$V(s'_1) \quad V(s'_2) \quad V(s'_3)$$

$$V(s) = \max_a (R(s,a) + \gamma V(s'))$$

Markov Decision Process (MDP)

$$0.8 * V(s'_1) + 0.1 * V(s'_2) + 0.1 * V(s'_3)$$

$$V(s) = \max_a (R(s,a) + \gamma V(s'))$$

Markov Decision Process (MDP)

$$V(s) = \max_a \left(R(s,a) + \gamma \sum_{s'} P(s,a,s')V(s') \right)$$

Policy vs Plan

Policy is mapping of actions with states in an environment with randomness.

Plan is also the same, but in a hypothetical deterministic environment without randomness.

